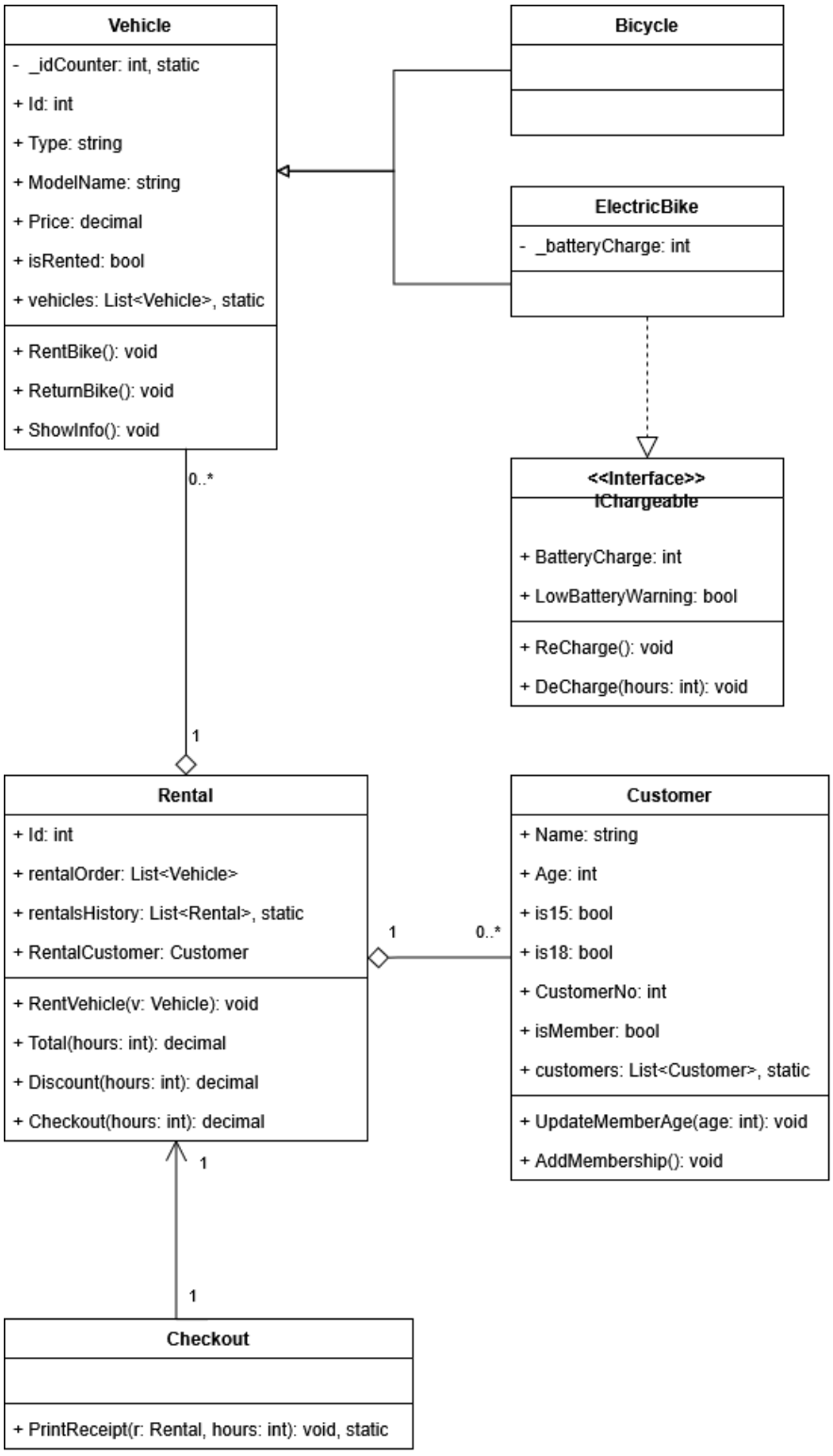
BED26GB
Jacob Horndahl

OOP1 – Individuell inlämningsuppgift – Kajens cykeluthyrning

UML diagram över programmet:



2026-10-29

BED26GB
Jacob Horndahl

Problem & buggar

En bugg som uppstod var i samband med implementering av interfacet IChargeable. Vid återlämning av cykel kraschade programmet när programmet skulle ta fram elcyklarna i uthyrningen för att kunna minska batterinivån. Om en vanlig cykel återlämnades kraschade programmet eftersom Bicycle klassen inte implementerar IChargeable och jag trodde att programmet skulle förstå detta och hoppa över de objekt som inte implementerade interfacet. Foreach-loopen behövde uppdateras med ”of type IChargeable”:

```
foreach (IChargeable ev in rentalToReturn.rentalOrder.OfType<IChargeable>())
{
   ev.DeCharge(rentedHours);
}
```

En annan bugg uppstod i sökfunktionen för att lämna tillbaks cykel baserat på ID-nummer. Eftersom jag valt att skapa ID-numret baserat på vilket år cykeln lades till, tex ”26001”, ville jag att det skulle gå att skriva in bara delar av numret och den skulle välja rätt. Problemet var att LINQ funktionen .Contains söker från början av strängen, så om man ville hyra cykel 26002 och bara skrev ”2” så valde den första cykeln i listan eftersom alla började med ”26”. Så istället för .Contains fick det ändras till .EndsWith. som letar från andra hållet:

```
string rentalIdSearchString = rentalIdChoice.ToString();
var choiceToRent = Vehicle.vehicles
   .Where(v => v.Id
   .ToString()
   .EndsWith(rentalIdSearchString))
   .FirstOrDefault();
```

En tredje bugg var en stack overflow vid uträkning av batterinivå i ElectricBike klassen:

```
public int BatteryCharge
{
  get
  {
    return Math.Max(0, BatteryCharge);
  }
```

Om man har med variabeln i sin egna getter blir det en evig loop. Koden fick justeras med en private variabel:

```
private int _batteryCharge;
public int BatteryCharge
{
  get
  {
    return Math.Max(0, _batteryCharge);
  }
```

Förbättringsmöjligheter

Dataförlust

Alla listor i programmet skapas när programmet kör och sparas inte när programmet stängs ner. Gäller:

- Cyklar
- Kunder
- Uthyrningar
- Uthyrningshistorik/kvitton

Alla listor borde sparas ner till fil, tex json, så att datan inte försvinner när programmet stängs.

Inaktuell data

Kundernas ålder bör baseras på DateTime födelseår så att rätt ålder alltid är aktuell via get-funktion som räknar ut ålder mot DateTime.Now.

Rigiditet

Cyklarnas priser är hårdkodade, metoder för att sätta och uppdatera priserna bör läggas in.

BED26GB
Jacob Horndahl

Automatiserad prisuträkning

Nu måste den anställde skriva in hur många timmar cykeln som återlämnas varit uthyrd. Vid uthyrning bör en DateTime.Now sparas så att antal timmar automatiskt uträknat till sista påbörjad timme kan användas.

Bokning av cyklar

Cyklar borde kunna bokas i förväg, samt att man bör ange hur länge cykeln är tänkt att vara uthyrd så det går att veta när cykeln kommer vara tillbaka och kan hyras ut igen.

Inlogg

Personalen borde ha inlogg så man vet vem som varit vid kassan.

Batteriladdning

Metoden för laddning av batteri sätter batteriet till 100% omedelbart, ”batteribyte”. En cykel kan nu också laddas även om den redan är uthyrd, vilken kan gå om kunden får ett nytt batteri i samband med uthyrningen, men kan bli fel om man ”laddar” cykeln när den är ute på stan.

I verkligheten kanske detta skulle kopplas till cykeln så den skickar data till hur mycket batteri som faktiskt finns. Samt en bool IsCharging kunnat vara användbar.

Kunduppgifter

Mer kunduppgifter borde samlas in för att kunna komma i kontakt med kunden.

2026-10-29